

# ANDY'S LINE MATCHER

## Stos dopasowania miast

Matcher dopasowuje miasta pomiędzy **shipment report** a **rate card**, liczy podobieństwo kandydatów i decyduje, czy para nadaje się do automatycznego złączenia, ręcznego review albo pozostawienia jako niedopasowana.

## Minimalne warunki pliku wejściowego

Plik wejściowy powinien spełniać podstawowy kontrakt danych:

- format pliku: **.xlsx**
- arkusze o nazwach:
  - **rc**
  - **shrep**
    - w arkuszu **rc** muszą istnieć kolumny:
      - **Destination City**
      - **Destination Country**
    - w arkuszu **shrep** muszą istnieć kolumny:
      - **Delivery City**
      - **Arrival Country**

Dodatkowe kolumny są dozwolone. Program ich nie wymaga do samego matchowania, ale zachowuje je w arkuszu wyjściowym **matched\_rows**.

Najbezpieczniejszy układ wejścia jest taki:

- nagłówki znajdują się w pierwszym wierszu
- dane są w jednej spójnej tabeli na arkusz
- nie ma kilku wierszy tytułowych nad tabelą
- nazwy wymaganych kolumn pasują dokładnie do oczekiwanej pisowni

Warto też pamiętać:

- miasta i kraje nie muszą być idealnie ujednolicone, bo matcher je normalizuje
- puste wartości są dozwolone, ale osłabiają szansę na poprawne dopasowanie
- jeśli zmieni się nazwa arkusza albo nagłówek wymaganej kolumny, program nie znajdzie danych automatycznie

## Po co istnieje ten matcher

---

W praktyce dane logistyczne rzadko są idealnie spójne:

- to samo miasto bywa zapisane na kilka sposobów
- jedne źródła używają myślników, przecinków albo skrótów, inne nie
- raz pojawia się sam rdzeń nazwy miasta, a innym razem dłuższa forma z regionem lub krajem
- czasem kolejność tokenów jest inna
- zdarzają się drobne literówki albo transliteracyjne warianty pisowni

Ten program robi tylko jedną rzecz:

- znajduje najbardziej prawdopodobne połączenie **Delivery City** z **shipment report** do **Destination City** z **rate card**

Czyli robimy porządne, audytowalne dopasowanie danych wejściowych, które można później bezpiecznie wykorzystać w dalszej pracy operacyjnej.

## Co ten program robi, a czego nie robi

---

Matcher jest **deterministyczny**. To znaczy:

- nie używa modelu ML
- nie ma etapu trenowania
- nie korzysta z embeddingów

- nie używa LLM-a do podejmowania decyzji
- dla tych samych danych i tych samych progów zawsze zwróci ten sam wynik

To ważne, bo w środowisku operacyjnym łatwiej wtedy:

- wyjaśnić decyzję
- odtworzyć wynik
- poprawić reguły
- odróżnić auto-match od przypadków wymagających review

## Pipeline dopasowania

---

Dla pojedynczego wiersza z `shipment report` program przechodzi przez następujące etapy:

1. normalizuje `Arrival Country`
2. normalizuje `Delivery City`
3. ogranicza pulę kandydatów tylko do RC z tym samym krajem docelowym
4. sprawdza, czy istnieje ręcznie zatwierdzony alias w `city_aliases.csv`
5. jeśli aliasu nie ma, liczy score fuzzy dla wszystkich kandydatów
6. zostawia najlepsze dopasowania po deduplikacji na poziomie wiersza RC
7. na podstawie progu i przewagi nad drugim kandydatem decyduje, czy wynik jest:

- `auto_matched`
- `review_needed`
- `unmatched`

To jest celowo warstwowe. Najpierw zawężamy problem logicznie, a dopiero potem uruchamiamy fuzzy scoring.

## Warstwa normalizacji

---

Normalizacja jest krytyczna, bo większość problemów z dopasowaniem nie wynika z "nieznanego miasta", tylko z innego zapisu tego samego miasta.

`normalize_basic`

To pierwszy i najbardziej ogólny poziom czyszczenia tekstu.

Funkcja:

- obcina białe znaki z początku i końca
- zamienia tekst na wielkie litery
- usuwa diakrytyki przez normalizację Unicode `NFKD`
- zamienia znaki niealfanumeryczne na spacje
- redukuje wielokrotne spacje do pojedynczej

Przykłady:

- `Dubai, UAE` -> `DUBAI UAE`
- `Petah-Tikva` -> `PETAH TIKVA`
- `São Paulo` -> `SAO PAULO`

Po co to robimy:

- eliminujemy różnice czysto techniczne
- ujednolicamy zapis niezależnie od formatu źródła
- przygotowujemy tekst do porównań znakowych i tokenowych

`normalize_city`

To warstwa bardziej wyspecjalizowana pod miasta.

Najpierw używa `normalize_basic`, a potem:

- usuwa ogólne słowa typu `CITY, TOWN, DISTRICT, PROVINCE`
- jeśli na końcu nazwy miasta występuje token odpowiadający krajowi i nadal zostaje sensowna nazwa miasta, odcina ten końcowy token

Przykłady:

- `Dubai, UAE` przy kraju `AE` -> `DUBAI`
- `Makati City` -> `MAKATI`
- `Sharjah UAE` przy kraju `AE` -> `SHARJAH`

To jest ważne, bo dane RC bardzo często mają formę "miasto + dopisek", a `shipment report` tylko rdzeń nazwy.

`city_variants`

W `rate card` pojedyncza komórka miasta potrafi zawierać kilka fragmentów lokalizacji naraz, na przykład:

- `BERKELEY VALE; WARNERVALE NSW`

Dlatego matcher nie zakłada, że w komórce RC istnieje tylko jedna nazwa. Funkcja:

- bierze znormalizowaną wersję całej komórki
- rozбивa tekst po separatorach:

- ;  
- ,  
- /  
- |

- normalizuje każdy fragment osobno
- zapisuje wszystkie warianty jako wyszukiwalne reprezentacje tego samego wiersza RC

Efekt jest taki, że jeden rekord RC może być reprezentowany przez kilka sensownych wariantów nazwy, ale nadal prowadzi do tego samego wiersza biznesowego.

## Filtrowanie po kraju

---

Zanim w ogóle policzymy fuzzy score, matcher zawęży pulę kandydatów do tych, dla których:

- `Arrival Country == Destination Country` po normalizacji

To jest bardzo mocny filtr bezpieczeństwa.

Dlaczego:

- to samo albo podobnie brzmiące miasto może istnieć w różnych krajach
- fuzzy matching bez filtra kraju zbyt łatwo daje fałszywie atrakcyjne wyniki
- kraj jest zwykle bardziej stabilnym atrybutem niż samo pole miasta

Jeśli kandydatów w tym kraju nie ma, kod potrafi technicznie zrobić fallback cross-country, ale domyślnie ta ścieżka jest wyłączona. To świadoma decyzja ostrożnościowa.

## Warstwa pamięci ręcznej: `city_aliases.csv`

---

`city_aliases.csv` pełni rolę pamięci operacyjnej. To nie jest model, tylko jawna tabela zatwierdzonych mapowań.

Jeśli istnieje rekord dla:

- znormalizowanego kraju źródłowego

- znormalizowanego miasta źródłowego

to matcher kończy pracę natychmiast i zwraca:

- `status = auto_matched`
- `method = alias`
- `score = 100`

bez uruchamiania fuzzy rankingu.

W praktyce daje to dwie korzyści:

- raz ręcznie rozwiązany przypadek nie wymaga ponownego zgadywania
- z czasem system uczy się operacyjnie przez jawne aliasy, ale nadal pozostaje w pełni audytowalny

## Silnik fuzzy scoringu

---

Jeżeli alias nie istnieje, każdemu kandydatowi przypisywany jest score liczony trzema heurystykami. Wynik końcowy to maksimum z tych trzech metod.

To bardzo ważny punkt: nie próbujemy zbudować jednego "sprytnego" wzoru na wszystko. Zamiast tego łączymy kilka prostych metod, z których każda dobrze łapie inny typ podobieństwa.

## 1. Gestalt similarity

---

Funkcja:

- `sequence_score(left, right)`

Implementacja:

- Python `difflib.SequenceMatcher`

Wzór:

```
score_seq = 100 * SequenceMatcher(None, left, right).ratio()
```

### Co to mierzy

To jest podobieństwo w stylu Ratcliff-Obershelp. Algorytm patrzy na dwa napisy i szuka dużych wspólnych bloków znaków. Im więcej wspólnej struktury i im mniej różnic, tym wyższy wynik.

W praktyce dobrze działa na:

- drobne literówki
- brak lub nadmiar jednej litery
- warianty typu KH vs H
- małe różnice w transliteracji

Przykłady:

- SHARJACH vs SHARJAH
- PETAH TIKVA vs PETAKH TIKVA
- BEIT DAGAN vs BET DAGAN

### Intuicja

Jeżeli dwa stringi wyglądają podobnie "gołym okiem", ta metoda zazwyczaj daje wysoki wynik. Jest bardzo dobra jako pierwszy, uniwersalny detektor podobieństwa znakowego.

## 2. Token sort similarity

---

Funkcja:

- `token_sort_score(left, right)`

Wzór:

1. rozbij oba stringi na tokeny po spacjach
2. posortuj tokeny alfabetycznie
3. sklej tokeny z powrotem w string
4. policz `sequence_score` na tych posortowanych wersjach

Formalnie:

```
sorted_left = " ".join(sorted(left.split()))
sorted_right = " ".join(sorted(right.split()))
score_token_sort = sequence_score(sorted_left, sorted_right)
```

### Co to mierzy

Ta metoda jest specjalnie pod przypadki, w których zawartość tekstowa jest podobna, ale kolejność słów została przestawiona.

Przykład:

- `PETAH TIKVA`
- `TIKVA PETAH`

Po sortowaniu oba ciągi stają się logicznie tym samym zestawem tokenów w tej samej kolejności, więc score rośnie.

### Dlaczego to jest potrzebne

Sam `SequenceMatcher` jest wrażliwy na kolejność znaków. Gdy słowa są poprawne, ale przestawione, zwykły score sekwencyjny może spaść bardziej niż powinien. Token sort niweluje ten efekt.

## 3. Subset / containment heuristic

---

Funkcja:

- `subset_score(left, right)`

Logika:

1. zamień oba stringi na zbiory tokenów
2. wybierz krótszy i dłuższy zbiór
3. jeśli krótszy zbiór w całości zawiera się w dłuższym, przyznaj wysoki score

Wzór:

```
if shorter is subset of longer:
    token_gap = len(longer) - len(shorter)
    score_subset = max(90, 97 - 2 * token_gap)
else:
    score_subset = 0
```

### Co to mierzy

Ta heurystyka łapie sytuacje, gdzie krótsza nazwa miasta jest rzeczywiście zawarta w dłuższej formie RC.

Przykłady:

- `WARNERVALE`
- `BERKELEY VALE WARNERVALE NSW`



albo:

- DUBAI
- DUBAI SOUTH FREE ZONE

### Dlaczego zwykły string similarity tu nie wystarcza

W takich przypadkach RC może mieć dużo dodatkowego kontekstu. `SequenceMatcher` nie zawsze da wtedy wynik tak wysoki, jak biznesowo byśmy chcieli. Heurystyka containment mówi wprost:

- jeśli cały rdzeń nazwy występuje w bogatszej formie, traktuj to jako mocną przesłankę

Jednocześnie score jest lekko obniżany wraz z rosnącą liczbą dodatkowych tokenów, żeby bardzo długie i zbyt szerokie opisy nie dostawały idealnego wyniku bez refleksji.

## Wynik końcowy fuzzy

---

Funkcja:

- `city_score(left, right)`

Wzór:

```
score_final = max(
    score_seq,
    score_token_sort,
    score_subset
)
```

### Dlaczego bierzemy maksimum

Każda z metod jest najlepsza na inny typ problemu:

- `sequence_score` na literówki i podobieństwo znakowe
- `token_sort_score` na przestawioną kolejność słów
- `subset_score` na skrócone nazwy zawarte w dłuższych opisach

Biorąc maksimum, nie uśredniamy sygnałów na siłę. Pozwalamy, żeby wygrała ta heurystyka, która najlepiej tłumaczy dany przypadek.

## Ranking i deduplikacja kandydatów

---

Jeden wiersz RC może wygenerować kilka wariantów przez `city_variants`. To oznacza, że ten sam rekord biznesowy mógłby pojawić się wielokrotnie w rankingu.

Żeby tego uniknąć:

- kandydaci są grupowani po `rc_row_id`
- dla każdego wiersza RC zostaje tylko najwyższy osiągnięty score

Potem kandydaci są sortowani malejąco po wyniku.

To jest ważne, bo użytkownik ma oceniać rzeczywiste alternatywy biznesowe, a nie trzy różne warianty tej samej komórki.

## Reguła decyzji

---

Po policzeniu rankingu bierzemy:

- `top_score` - najlepszy wynik
- `second_score` - drugi najlepszy wynik albo 0
- `margin = top_score - second_score`

Logika decyzji:

```
if top_score >= auto_threshold and (top_score == 100 or margin >= min_margin):
    status = auto_matched
elif top_score >= review_threshold:
    status = review_needed
else:
    status = unmatched
```

Domyślne progi:

- `auto_threshold = 90`
- `review_threshold = 75`
- `min_margin = 3`

### Jak to interpretować

Sam wysoki score nie zawsze wystarcza. Jeśli pierwszy i drugi kandydat są niemal identyczni punktowo, to znaczy, że matcher widzi niepewność. Wtedy:

- wynik nie powinien wpadać automatycznie
- lepiej skierować przypadek do manual review

`margin` jest więc prostą miarą przewagi najlepszego kandydata nad resztą stawki.

To ważne operacyjnie, bo odróżnia dwa przypadki:

- "ten kandydat naprawdę wygląda najlepiej"
- "ten kandydat jest najlepszy, ale tylko minimalnie"

## Co trafia do review

---

Status `review_needed` oznacza:

- program widzi sensownego kandydata
- ale nie ma jeszcze wystarczającej pewności, żeby samemu zatwierdzić match

W konsoli użytkownik dostaje wtedy:

- najlepsze kandydatury
- metodę
- score

Po ręcznej akceptacji taka para może później trafić do `city_aliases.csv`, dzięki czemu następnym razem stanie się już przypadkiem prostym i automatycznym.

## Dlaczego to dobrze działa na danych logistycznych

---

Ten stack jest dobrze dopasowany do typowych problemów w danych transportowych:

- różne transliteracje
- literówki
- warianty z myślnikami, przecinkami i dopiskami
- nazwy z regionem albo krajem
- wiele lokalizacji wpisanych do jednej komórki
- przestawiona kolejność tokenów

Jednocześnie nie robi "magii", której nie da się później wyjaśnić.

Każdy match można rozebrać na czynniki:

- jaka była znormalizowana nazwa źródłowa

- jacy byli kandydaci
- która heurystyka dała najwyższy sygnał
- jaki był score
- jaka była przewaga nad drugim kandydatem

## Ograniczenia i świadome kompromisy

---

To rozwiązanie jest celowo lekkie, szybkie i przewidywalne, ale ma też granice.

Nie jest to:

- Levenshtein
- phonetic matching
- semantyczny matcher nazw
- embedding similarity
- machine learning

W konsekwencji słabiej poradzi sobie z przypadkami, gdzie:

- dwa miasta są bardzo podobne fonetycznie, ale mało podobne znakowo
- różne języki używają zupełnie innych egzonimów
- poprawne dopasowanie wymaga wiedzy geograficznej spoza samego tekstu

To jest akceptowalny kompromis, bo nasz priorytet to:

- prostota
- transparentność
- mała liczba zależności
- łatwość uruchomienia jako lekkie `exe`

## Podsumowanie architektury matematycznej

---

Cały "math stack" można zapisać tak:

1. normalizacja tekstu
2. ekspansja wariantów RC
3. filtr po kraju
4. override przez alias

5. trzy równoległe heurystyki fuzzy:

- sekwencyjna
- token sort
- subset / containment

6. `score_final = max(...)`

7. ranking po `rc_row_id`

8. decyzja przez próg i margin

To nie jest system uczony na danych. To jest deterministyczny fuzzy matcher z pamięcią aliasów i kontrolowanym routingiem do review.